

PRACTICAL 8

Aim

To implement stock market analytics using Window-Based Analysis in Apache Spark with PySpark.

The practical focuses on loading real historical stock market data obtained from Yahoo Finance into a Spark DataFrame, examining its schema and records, grouping stock data by stock symbol, calculating average, maximum and minimum stock prices, applying Spark Window functions, calculating running averages and moving averages, determining daily stock returns and volatility, comparing stock performance, generating a summary report, and visualizing stock price trends.

Problem Definition

A financial services company processes large volumes of historical stock market data collected from multiple stock exchanges. Investors and analysts need to understand stock price movements, compare different stocks, identify trends, and analyze their performance over time.

As the volume of financial data increases, traditional data processing methods may become inefficient for performing repeated calculations and time-based analysis. Apache Spark provides distributed DataFrame operations and Window functions that can efficiently perform analytical calculations over large datasets.

The task is to load real historical stock market data from Yahoo Finance for multiple stocks into a Spark DataFrame and perform window-based stock market analysis.

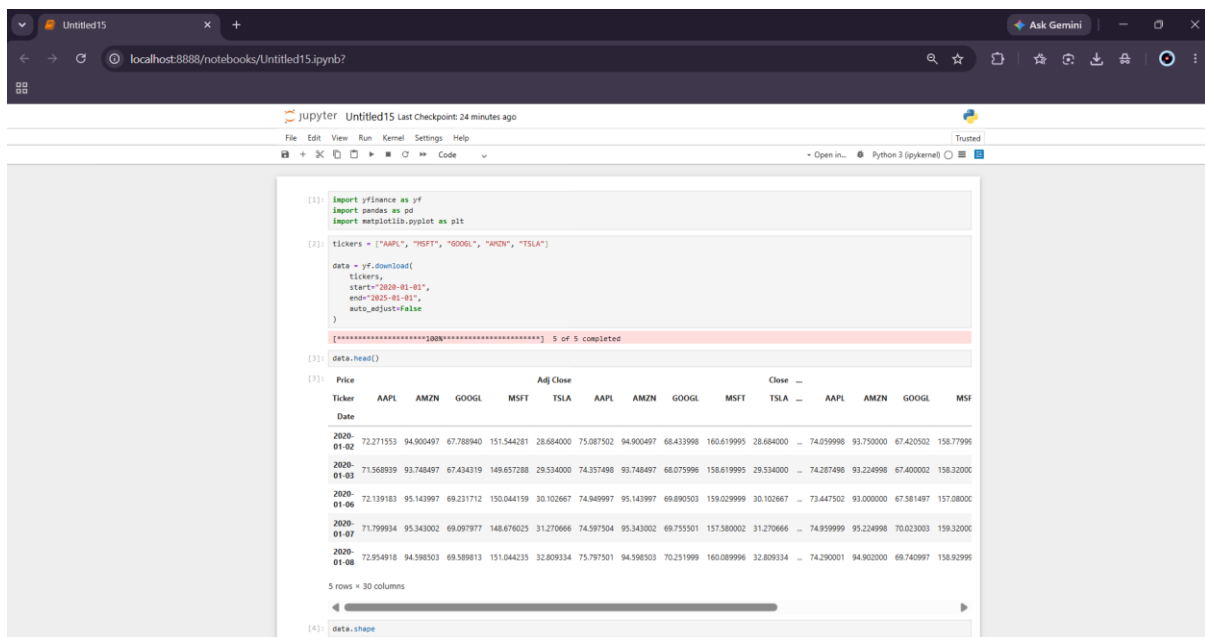
The practical includes creating a SparkSession, loading and displaying the stock dataset, examining the schema, grouping records based on stock symbols, calculating average, maximum and minimum stock prices, creating Window specifications, calculating running averages and 3-day moving averages, calculating daily stock returns using lag(), measuring stock volatility, comparing total stock returns, generating a stock performance summary report, and visualizing stock price trends.

Objectives

- Understand stock market analytics using Apache Spark.
- Create and configure a SparkSession.
- Obtain real historical stock market data using Yahoo Finance.
- Load multiple stock records into a Spark DataFrame.
- Display and examine the stock dataset.
- Display the schema of the DataFrame.
- Group stock records based on stock symbol.
- Calculate average stock price.
- Calculate maximum stock price.

- Calculate minimum stock price.
- Create Window specifications in Spark.
- Calculate running average stock prices.
- Calculate 3-day moving averages.
- Use lag() to obtain previous stock prices.
- Calculate daily stock returns.
- Calculate stock volatility.
- Calculate total stock returns.
- Compare the performance of multiple stocks.
- Generate a stock market summary report.
- Visualize historical stock price trends.
- Understand the importance of Window functions in time-series analysis.
- Analyze stock market trends using Spark.

Output



The image displays two sequential screenshots of a Jupyter Notebook environment, likely running on a local host (localhost:8888). The interface includes a browser window at the top, a Jupyter Notebook header, and a code editor with a Python kernel.

Top Screenshot: The notebook shows the initial data loading and inspection. The code cell [4] contains `data.shape`, which returns `(1258, 38)`. The code cell [5] contains `data.info()`, which displays the following information:

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1258 entries, 2020-01-02 to 2024-12-31
Data columns (total 38 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   Adj Close, AAPL     1258 non-null   float64
 1   Adj Close, AMZN     1258 non-null   float64
 2   Adj Close, GOOGL    1258 non-null   float64
 3   Adj Close, MSFT     1258 non-null   float64
 4   Adj Close, TSLA     1258 non-null   float64
 5   Close, AAPL         1258 non-null   float64
 6   Close, AMZN         1258 non-null   float64
 7   Close, GOOGL        1258 non-null   float64
 8   Close, MSFT         1258 non-null   float64
 9   Close, TSLA        1258 non-null   float64
10   High, AAPL          1258 non-null   float64
11   High, AMZN          1258 non-null   float64
12   High, GOOGL         1258 non-null   float64
13   High, MSFT          1258 non-null   float64
14   High, TSLA          1258 non-null   float64
15   Low, AAPL           1258 non-null   float64
16   Low, AMZN           1258 non-null   float64
17   Low, GOOGL          1258 non-null   float64
18   Low, MSFT           1258 non-null   float64
19   Low, TSLA           1258 non-null   float64
20   Open, AAPL          1258 non-null   float64
21   Open, AMZN          1258 non-null   float64
22   Open, GOOGL         1258 non-null   float64
23   Open, MSFT          1258 non-null   float64
24   Open, TSLA          1258 non-null   float64
25   Volume, AAPL        1258 non-null   int64  
26   Volume, AMZN        1258 non-null   int64  
27   Volume, GOOGL       1258 non-null   int64  
28   Volume, MSFT       1258 non-null   int64  
29   Volume, TSLA       1258 non-null   int64  
dtypes: float64(25), int64(5)
memory usage: 384.7 KB
```

The code cell [6] contains `close_data = data["Close"]` and `close_data.head()`, which displays the following data:

	Ticker	Adj Close
2020-01-02	AAPL	129.01
2020-01-02	AMZN	184.00
2020-01-02	GOOGL	2864.00
2020-01-02	MSFT	160.61
2020-01-02	TSLA	248.48

Bottom Screenshot: The notebook shows the data cleaning and reshaping process. The code cell [6] contains `close_data = data["Close"]` and `close_data.head()`, which displays the following data:

	Ticker	Adj Close
2020-01-02	AAPL	129.01
2020-01-02	AMZN	184.00
2020-01-02	GOOGL	2864.00
2020-01-02	MSFT	160.61
2020-01-02	TSLA	248.48

The code cell [7] contains `stock_data = close_data.stack().reset_index()`. The code cell [8] contains `stock_data.head()`, which displays the following data:

	Date	Ticker	Adj Close
0	2020-01-02	AAPL	129.01
1	2020-01-02	AMZN	184.00
2	2020-01-02	GOOGL	2864.00
3	2020-01-02	MSFT	160.61
4	2020-01-02	TSLA	248.48

The code cell [9] contains `stock_data.columns = ["Date", "Stock", "Price"]`. The code cell [10] contains `stock_data.head(10)`, which displays the following data:

	Date	Stock	Price
0	2020-01-02	AAPL	129.01
1	2020-01-02	AMZN	184.00
2	2020-01-02	GOOGL	2864.00
3	2020-01-02	MSFT	160.61
4	2020-01-02	TSLA	248.48

A Snipping Tool window is visible in the bottom right corner, displaying the text: "Screenshot copied to clipboard. Automatically saved to screenshots folder. Mark-up and share".

The image displays two screenshots of a Jupyter Notebook interface, likely running on a local host (localhost:8888/notebooks/Untitled15.ipynb). The notebook is titled "Untitled15" and shows a series of code cells for data analysis.

Top Screenshot:

- Code cell [10]: `stock_data.head(10)` displays the first 10 rows of the stock data.
- Code cell [11]: `stock_data = stock_data.dropna()` removes rows with missing values.
- Code cell [12]: `stock_data.isnull().sum()` checks for null values in each column.
- Code cell [13]: `stock_data["Stock"].unique()` displays the unique stock symbols.
- Code cell [14]: `stock_data["Stock"].value_counts()` displays the count of each stock symbol.

Bottom Screenshot:

- Code cell [15]: `stock_data.head()` displays the first 5 rows of the stock data.
- Code cell [16]: `from pyspark.sql import SparkSession` imports the SparkSession class.
- Code cell [17]: `spark = SparkSession.builder \` creates a SparkSession object.
- Code cell [18]: `stock_df = spark.createDataFrame(stock_data)` creates a DataFrame from the stock data.
- Code cell [19]: `stock_df.show(10)` displays the first 10 rows of the DataFrame.

The bottom screenshot also shows a "Snipping Tool" window in the background, indicating a screenshot was taken.

The image displays two screenshots of a Jupyter Notebook interface, showing PySpark code execution for stock data analysis. The notebook is titled "Untitled15" and is running on a local host (localhost:8888/notebooks/Untitled15.ipynb?).

Top Screenshot:

- Code cell [20]: `stock_df.printSchema()`
- Code cell [21]: `print("Total Records:", stock_df.count())`
- Code cell [22]: `print("Total Stocks:", stock_df.select("Stock").distinct().count())`
- Code cell [23]: `stock_df.groupby("Stock").count().show()`
- Code cell [24]: `from pyspark.sql.functions import avg`
`avg_price = stock_df.groupby("Stock").agg(avg("Price").alias("Average_Price"))`
`avg_price.show()`

Bottom Screenshot:

- Code cell [25]: `from pyspark.sql.functions import max`
`max_price = stock_df.groupby("Stock").agg(max("Price").alias("Maximum_Price"))`
`max_price.show()`
- Code cell [26]: `from pyspark.sql.functions import min`
`min_price = stock_df.groupby("Stock").agg(min("Price").alias("Minimum_Price"))`
`min_price.show()`
- Code cell [27]: `from pyspark.sql.functions import avg, max, min`
`stock_summary = stock_df.groupby("Stock").agg(avg("Price").alias("Average_Price"), max("Price").alias("Maximum_Price"), min("Price").alias("Minimum_Price"))`
`stock_summary.show()`

The screenshot displays a Jupyter Notebook interface with the following code and output:

```
stock_summary.show()

[27]: from pyspark.sql.window import Window

[28]: window_spec = Window \
      .partitionBy("Stock") \
      .orderBy("Date")

[29]: from pyspark.sql.functions import avg

      running_avg_df = stock_df.withColumn(
          "Running_Average",
          avg("Price").over(window_spec)
      )

      running_avg_df.show(10)
```

Date[Stock]	Price	Running_Average
2020-01-02 00:00:00 AAPL	75.0875015258789	75.0875015258789
2020-01-03 00:00:00 AAPL	74.35740816894531	74.72249984741211
2020-01-06 00:00:00 AAPL	74.9499969424219	74.798322435547
2020-01-07 00:00:00 AAPL	75.5575036213938	74.7461250749396
2020-01-08 00:00:00 AAPL	75.75750061835156	74.9580081830547
2020-01-09 00:00:00 AAPL	77.48750122870312	75.362503568384
2020-01-10 00:00:00 AAPL	77.52424966438644	75.482850868789
2020-01-13 00:00:00 AAPL	79.23999786376953	76.12749580833
2020-01-14 00:00:00 AAPL	78.2899918794531	76.34443880023
2020-01-15 00:00:00 AAPL	77.8349998447266	76.3824993894843

only showing top 10 rows

```
[30]: moving_window = Window \
      .partitionBy("Stock") \
      .orderBy("Date") \
      .rowsBetween(-2, 0)
```

```
[31]: moving_avg_df = stock_df.withColumn(
      "Moving_Average_3_Day",
      avg("Price").over(moving_window)
  )

      moving_avg_df.show(10)
```

Date[Stock]	Price	Moving_Average_3_Day
2020-01-02 00:00:00 AAPL	75.0875015258789	75.0875015258789
2020-01-03 00:00:00 AAPL	74.35740816894531	74.72249984741211
2020-01-06 00:00:00 AAPL	74.9499969424219	74.798322435547
2020-01-07 00:00:00 AAPL	75.575036213938	74.6349999308095
2020-01-08 00:00:00 AAPL	75.75750061835156	75.1150000690205
2020-01-09 00:00:00 AAPL	77.48750122870312	75.93416849772136
2020-01-10 00:00:00 AAPL	77.52424966438644	76.8291681580486
2020-01-13 00:00:00 AAPL	79.23999786376953	76.4766562551302
2020-01-14 00:00:00 AAPL	78.2899918794531	76.33683889192708
2020-01-15 00:00:00 AAPL	77.8349998447266	76.4169837239583

only showing top 10 rows

```
[32]: from pyspark.sql.functions import lag

      returns_df = stock_df.withColumn(
          "Previous_Price",
          lag("Price", 1).over(window_spec)
      )

      returns_df.show(10)
```

Date[Stock]	Price	Previous_Price
2020-01-02 00:00:00 AAPL	75.0875015258789	NULL
2020-01-03 00:00:00 AAPL	74.35740816894531	75.0875015258789
2020-01-06 00:00:00 AAPL	74.9499969424219	74.35740816894531
2020-01-07 00:00:00 AAPL	75.575036213938	74.9499969424219
2020-01-08 00:00:00 AAPL	75.75750061835156	75.575036213938
2020-01-09 00:00:00 AAPL	77.48750122870312	75.75750061835156
2020-01-10 00:00:00 AAPL	77.52424966438644	77.48750122870312
2020-01-13 00:00:00 AAPL	79.23999786376953	77.52424966438644
2020-01-14 00:00:00 AAPL	78.2899918794531	79.23999786376953
2020-01-15 00:00:00 AAPL	77.8349998447266	78.2899918794531

only showing top 10 rows

The interface also shows a Windows taskbar at the bottom with the date 30-09-2026 and time 19:22. A Snipping Tool notification is visible on the right side of the notebook, indicating a screenshot was copied to the clipboard.

The image displays two screenshots of a Jupyter Notebook interface, likely running on a local host (localhost:8888/notebooks/Untitled15.ipynb). The notebook is titled "Untitled15" and shows code executed in a Python 3 (ipykernel) environment.

Top Screenshot:

Code cell [33]:

```
from pyspark.sql.functions import round

returns_df = returns_df.withColumn(
    "Daily_Return_Percent",
    round(
        (returns_df["Price"] - returns_df["Previous_Price"])
        / returns_df["Previous_Price"],
        2
    )
)

returns_df.show(10)
```

Output:

Date[Stock]	Price	Previous_Price	Daily_Return_Percent
2020-01-02 00:00:00	ASP1 75.087501258789	NULL	NULL
2020-01-03 00:00:00	ASP1 74.35749816894531	75.087501258789	-0.97
2020-01-06 00:00:00	ASP1 74.94999846232374	74.35749816894531	0.8
2020-01-07 00:00:00	ASP1 74.50750366238938	74.94999846232374	-0.47
2020-01-08 00:00:00	ASP1 75.70750063755167	74.50750366238938	1.61
2020-01-09 00:00:00	ASP1 77.48750122070322	75.70750063755167	2.12
2020-01-10 00:00:00	ASP1 77.58249664306647	77.48750122070322	0.23
2020-01-13 00:00:00	ASP1 79.23999786376953	77.58249664306647	2.14
2020-01-14 00:00:00	ASP1 78.16999816894531	79.23999786376953	-1.35
2020-01-15 00:00:00	ASP1 77.83499804472667	78.16999816894531	-0.43

Code cell [34]:

```
from pyspark.sql.functions import stddev

volatility_df = returns_df.groupBy("Stock").agg(
    stddev("Daily_Return_Percent").alias("Volatility")
)

volatility_df.show()
```

Output:

Stock	Volatility
ASP1	1.35

Bottom Screenshot:

Code cell [35]:

```
from pyspark.sql.functions import first, last

performance_df = stock_df.groupBy("Stock").agg(
    first("Price").alias("Starting_Price"),
    last("Price").alias("Ending_Price")
)

performance_df.show()
```

Output:

Stock	Starting_Price	Ending_Price
ASP1	75.087501258789	250.43999816894531
TSLA	28.68400081258789	403.839996378906
GOOGL	48.43399818791816	189.3000038517578
AMZN	190.04073465234412	389.99993064064
MSFT	168.6199951171875	421.5

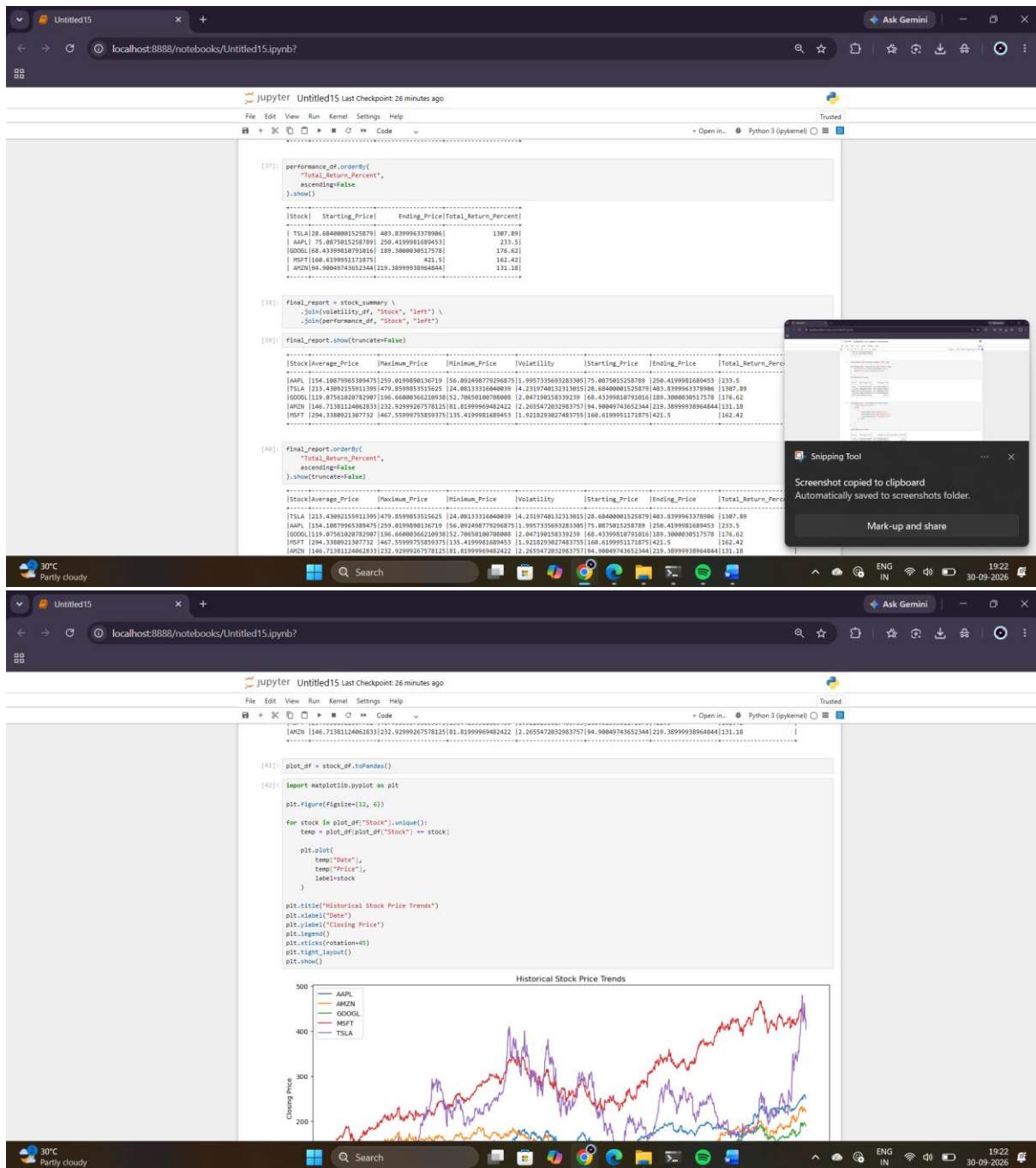
Code cell [36]:

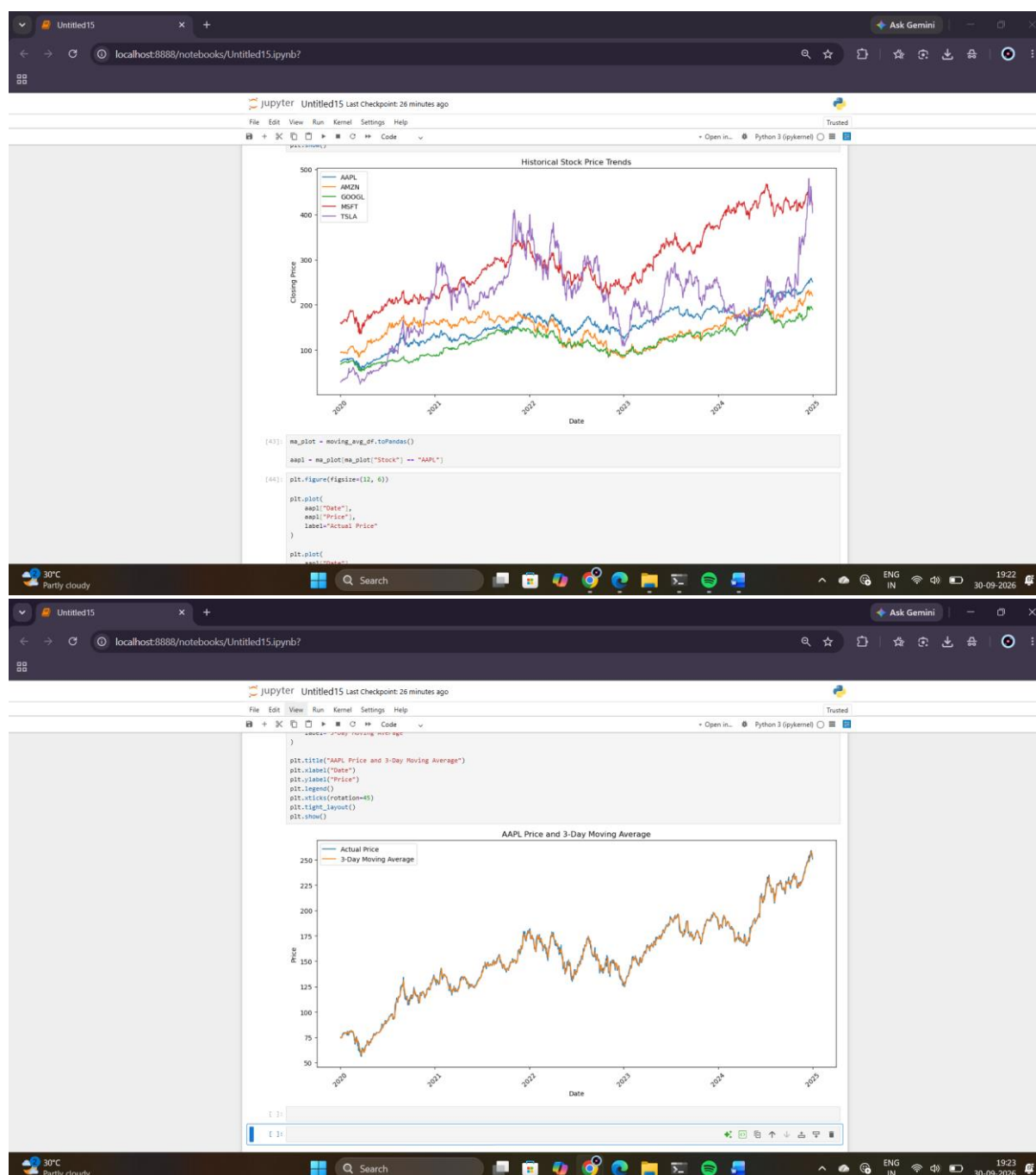
```
performance_df = performance_df.withColumn(
    "Total_Return_Percent",
    round(
        (performance_df["Ending_Price"] -
         performance_df["Starting_Price"])
        / performance_df["Starting_Price"],
        2
    )
)

performance_df.show()
```

Output:

Stock	Starting_Price	Ending_Price	Total_Return_Percent
ASP1	75.087501258789	250.43999816894531	233.51
TSLA	28.68400081258789	403.839996378906	1387.89
GOOGL	48.43399818791816	189.3000038517578	176.62
AMZN	190.04073465234412	389.99993064064	153.10
MSFT	168.6199951171875	421.5	162.42





Key Questions / Analysis

Q1. Why is Apache Spark suitable for financial analytics?

Apache Spark is suitable for financial analytics because it can process large volumes of financial data efficiently using distributed processing.

Spark provides DataFrames, aggregation functions and Window functions that are useful for analyzing historical stock prices. It can perform calculations such as average price, maximum price, minimum price, moving averages, daily returns and volatility.

Spark can also process data in parallel, making it suitable for large-scale financial datasets.

Q2. What is the purpose of grouping stock data by stock symbol?

Grouping stock data by stock symbol separates the records of each individual stock.

For example, stocks such as AAPL, MSFT, GOOGL, AMZN and TSLA can be analyzed separately.

Grouping allows us to calculate statistics such as:

1. Average stock price
2. Maximum stock price
3. Minimum stock price
4. Stock volatility
5. Total return

In PySpark, grouping can be performed using:

```
stock_df.groupBy("Stock").count().show()
```

Q3. How does average stock price help investors?

Average stock price provides an overall measure of the typical price of a stock during a selected period.

It helps in understanding the general price level of the stock and allows investors to compare the historical price levels of different stocks.

However, average price alone does not indicate whether a stock is a good investment because other factors such as returns, volatility, company performance and market conditions also need to be considered.

Q4. Why are maximum and minimum prices important?

Maximum and minimum prices show the highest and lowest prices recorded by a stock during a particular period.

They help in understanding:

1. The historical price range.
2. Price fluctuations.
3. High and low price levels.

4. Changes in stock prices over time.
5. The overall movement of the stock.

These values can also be used along with other metrics to analyze historical stock trends.

Q5. How can Spark support real-time stock market monitoring?

Apache Spark can support stock market monitoring by processing continuously generated market data and performing analytical operations on incoming records.

Spark can be used to:

1. Process large volumes of incoming stock data.
2. Calculate price changes and returns.
3. Track moving averages.
4. Detect unusual price movements.
5. Calculate volatility.
6. Compare stock performance.
7. Generate analytical results for monitoring systems.

For real-time applications, Spark's Structured Streaming can process continuously arriving data, allowing analytical calculations to be performed as new market records are received.

Supplementary Problems

1. Calculate daily stock returns.
2. Identify top-performing stock over a period.
3. Compute moving average for stock prices.
4. Compare stock volatility across companies.
5. Visualize stock price trends using charts.

Conclusion

The Stock Market Analytics Using Window-Based Analysis practical was successfully completed using PySpark and real historical stock market data obtained from Yahoo Finance.

The stock data for multiple companies was loaded into a Spark DataFrame and its records and schema were examined. The data was grouped according to stock symbols, and average, maximum and minimum stock prices were calculated.

Spark Window functions were then used for time-series analysis. Running averages and 3-day moving averages were calculated to analyze stock price trends. The lag() function was used to obtain previous stock prices, which enabled the calculation of daily stock returns.

Stock volatility was calculated using the standard deviation of daily returns, and starting price, ending price and total return percentage were used to compare the historical performance of different stocks. A final summary report was generated containing the major stock performance metrics.

The practical demonstrated how Spark DataFrames, aggregation functions, Window functions, lag(), moving averages and distributed processing can be used for analyzing large-scale financial datasets and identifying historical stock market trends.